Review retrospective:
2D pure-quartic affine covariance

Daniel Murfet (with Claude Opus 4.8 and GPT-5.5 Pro)

2026-05-31

Abstract

The first *Tide-review* excursion: an independent second-eye pass over an already-landed Lean file, `Laplace/TwoD/PureQuartic.lean`, which formalises the affine covariance of the doubly-degenerate 2D Gibbs measure with potential $L(x,y) = (x^4 + y^4)/24$. Where a Tide extends the shoreline and a Tide-experiment validates a formal value against numerics, a Tide-review turns back over taken ground and asks an independent model two questions: does the Lean faithfully match its informal content, and can it be simplified without changing any public statement. GPT-5.5 Pro reviewed the file against its module docstring and the 2 May tide retrospective and returned a *clean fidelity pass* — no wrong constant, exponent, sign, or hypothesis on the headline theorem or the moment table — together with five signature-preserving simplifications and three signature-*changing* suggestions correctly held back as future-tide work. Two simplifications were implemented under the review discipline's two invariants (green build, byte-identical public signatures), affirmed by a second GPT pass, and committed: the file drops from 478 to 454 lines with all 14 public declarations unchanged and zero `sorry`/`axiom`. The run also debugged the new `tide-review` skill itself, surfacing two process refinements folded back into it.

1 Setting

This is the prototype run of a new member of the Tide family. The `tide`, `tide-experiment`, `tide-pick`, and `tide-publish` skills all face *outward* — they take new ground, validate it, or publish it. The `tide-review` skill faces *inward*: it subjects landed, verified Lean to independent review by GPT-5.5 Pro. The model diversity is the mechanism. Claude wrote the Lean originally; GPT is the genuinely independent eye, and GPT also adjudicates the resulting fix.

The defining hazard, and the rule that contains it, is that a review *mutates already-verified proofs*. "The build is still green" is therefore necessary but not sufficient: a green build proves you proved *something*, not that you proved the thing the paper meant. The skill splits findings accordingly. *Fidelity* findings — a definition or statement that does not match the informal content — are correctness findings, reported and escalated to the human, never silently rewritten. *Simplification* findings — shorter proofs, better reuse, dead code — are meaning-preserving and may be implemented, but only under two invariants: the library still builds, and the set of public declaration signatures is *byte-identical* before and after. A change that moves a signature is, by definition, not a simplification; it is either a fidelity edit (escalate) or an API change (out of scope for a review — spin a forward tide).

The target was chosen as a well-anchored single file: the 2D pure-quartic covariance landed on 2 May as the fifth excursion in the `laplace` seabed,[1] and carries both a thorough module docstring

[1] Reviewed at `laplace` commit `a18980c`.

and a dedicated tide retrospective,[2] so the "informal content" the fidelity check compares against is unusually complete.

2 What was reviewed

`Laplace/TwoD/PureQuartic.lean`: 14 public declarations building to

> *For $L(x, y) = x^4/24 + y^4/24$, all $t > 0$, and affine observables $\varphi = a_1x + a_2y + c$, $\psi = b_1x + b_2y + d$,*
>
> $$\mathrm{Cov}_t[\varphi, \psi] = (a_1b_1 + a_2b_2)\sqrt{24/t}\,\Gamma(3/4)/\Gamma(1/4).$$

The informal corpus assembled for the reviewer: the file's own module docstring (which states this closed form, the Boltzmann factorisation, and the architecture), the 2 May retrospective (same closed form, the proof strategy, and a `scipy.dblquad` sanity check agreeing to 1.4×10^{-14}), and the seabed `CLAUDE.md`. There is no primer-paper `\leanref` anchor — this is a tide excursion beyond the primer core — so the docstring and retrospective *are* the whole informal ground truth.

3 Fidelity findings

None. This is the headline result. GPT-5.5 Pro compared every public statement against the docstring and retrospective and found no mismatch: `pureQuarticPotential` is exactly $x^4/24 + y^4/24$; `partitionFunction_closed_form` matches the prose $Z_{2D} = ((1/2)(24/t)^{1/4}\Gamma(1/4))^2$; the moment table (zero first moments, zero mixed moment, equal second moments) matches; and `cov_affine_pureQuartic` matches the headline verbatim, with no wrong constant, exponent, factor, sign, or hypothesis.

> **GPT-5.5 Pro:** I do not see any wrong constant, wrong exponent, missing factor, wrong sign, or mismatch in hypotheses on the headline result. The only "stronger than mathematically necessary" assumptions are on some parity-only helper lemmas, but those are best treated as API choices for future cleanup rather than fidelity defects.

A clean fidelity pass is a publishable result in its own right: it is independent evidence, from a different model, that the formalisation says what the mathematics says. It is exactly the assurance a reviewer of the eventual paper would want and could not otherwise get.

4 Simplifications applied

Of five signature-preserving simplifications GPT proposed, two were implemented in this run (the other three were deferred to keep the prototype bounded; see Follow-ups). Both keep all 14 public signatures byte-identical.

[2] `2026-05-02-tide-2d-pure-quartic`.

S1 — odd-moment collapse. Three helper blocks proving $\int x\, e^{-tx^4/24}\, dx = 0$ each carried a ten-line extensional rewrite (`heq` via `ext/congr/ring`) before applying `quartic_moment_odd`. These collapsed to a single `simpa [mul_div_assoc] using Laplace.OneD.quartic_moment_odd 0 t`.

The `mul_div_assoc` is load-bearing and was *not* in GPT's first suggestion. The 1D lemma states the integrand as $e^{-(t\, x^4/24)}$, which Lean parses as $-((t\, x^4)/24)$; the goal, after the `@[simp] quarticPotential_apply` rewrites `quarticPotential` $x \mapsto x^4/24$, is $-(t\,(x^4/24))$. The two differ by associativity of $\cdot$ over $/$, which is precisely what the original block's `congr 2; ring` had been reconciling. Plain `simpa using ...` fails with a `has type ... but is expected to have type ...` mismatch on exactly that parenthesisation; adding `mul_div_assoc` to the simp set bridges it. This is the review loop's implementer step behaving as intended: the recommendation is not rubber-stamped, it is compiled, and where the reviewer's tactic is slightly wrong the implementer repairs it.

S4 — numerator-vanishing helper. The headline proof repeated, five times, the same three-line argument: a vanishing Gibbs expectation $\langle\cdot\rangle = 0$ implies the corresponding numerator integral vanishes, because the partition function is nonzero (`div_eq_zero_iff` discharged with `hZne`). These were hoisted into one local helper

$$\texttt{num\_zero} : \forall f,\ \langle f\rangle = 0 \to \textstyle\int_{\mathbb{R}^2} f(z)\, e^{-tL(z)} = 0,$$

reused at all five call sites (means, mixed moment) via `num_zero _ (gibbsExpectation_fst ht)` and siblings. Beta-reduction on the instantiated f matches each site's goal.

Gates. Full library build green (8304 jobs); the 14-declaration signature fingerprint is unchanged; no `sorry`, `axiom`, or `native_decide`; 478 $\to$ 454 lines ($-44/+20$). A second GPT-5.5 Pro pass on the diff affirmed both changes ("faithful, semantics-preserving simplifications with no apparent downside") and confirmed the deferred three were safely deferrable.

5 Disagreements and open questions

None requiring adjudication. The fidelity pass was clean, and the affirmation pass agreed with both implemented changes, so no finding had to be escalated to the human as a Claude/GPT split. The only items handed back are informational:

- Three *signature-changing* suggestions, correctly held out of the review and recorded for a future forward tide (below). GPT independently re-derived two of them — the unused `0 < t` on the parity-only lemmas and the generic separable-potential refactor — which the 2 May retrospective had *already* flagged as follow-ups. Two models, a month apart, converging on the same cleanup is a good sign the cleanup is real.

6 Lessons

- *Lazy worktree creation.* The skill initially created the review worktree at preflight. But a review is read-only until simplifications are confirmed, and a fidelity-only outcome never mutates code at all — so the $\sim$40 s Mathlib-cache cost is wasted for any review that yields no auto-edits. Refinement, now in the skill: claim the (cheap) ledger entry at preflight, but create the worktree lazily, only once triage produces a non-empty simplification bucket.

- *GPT consult plumbing.* `query_gpt.py` lives at repo-root `scripts/`, not in the skill folder, and must run under `uv run` (system Python lacks `openai`). A one-line note now sits in the skill's consult step.
- *A clean review is a result, not a null run.* The instinct to treat "found nothing wrong" as a non-event is wrong here: an independent fidelity-clean verdict from a second model is the deliverable a paper reviewer would value most. The retrospective leads with it.

7 Follow-ups

- *Forward tide (signature-changing):* drop the unused `0 < t` hypothesis from the three parity-only lemmas (`gibbsExpectation_fst/_snd/_fst_mul_snd`); weaken `gibbsExpectation_one` to a `Z` $\neq$ `0` hypothesis; and the larger refactor onto the generic `AddSeparable` separable-potential API, after which this file (and `SemiDegenerate`) specialise from one abstraction. All three change public signatures, so they are forward-tide work, not review work.
- *Later review pass:* the three deferred signature-preserving simplifications (two further `simpa` collapses on the `hI..'` integrability normalisations, and a `partitionFunction` definitional cleanup in the second-moment lemmas).
- *Skill / infrastructure:* wire the `-review-` slug route into `tide-mirror` and add the hand-preserved *Review* tab so this retrospective publishes alongside Tides and Experiments; add a `\reviewref` macro to the retrospective template.

Acknowledgements

GPT-5.5 Pro performed both the independent review and the affirmation pass; its full responses are preserved in the review log.[3]

[3] `projects/primer/review-log/2026-05-31-review-2d-pure-quartic.md` and the `gpt_responses/` alongside it.